ProjectBoard

PRODUCT PROPOSAL & REQUIREMENTS

Visual project management, self-hosted and white-label.

A modern Kanban-style workspace for teams — boards, cards, and collaboration that runs entirely on your infrastructure.

Prepared for	Prepared by	Version
[Client name]	**[Your company]**	**1.0 — 9 October 2026**

Contents

ProjectBoard is a placeholder product name, to be replaced with the client's brand.

00 Product Overview 3
1. What ProjectBoard Is 3
2. Business Value 3
3. Who It Serves 3
4. High-Level Scope 3
5. Document Set 4

01 Requirements 5
1. Functional Requirements 5
2. Non-Functional Requirements 7

02 System Architecture 9
1. High-Level Design 9
2. Technology Stack 9
3. Data Model (core entities) 10
4. API Surface 11
5. Security Architecture 11
6. Build & Release 12

03 UI / UX Specification 13
1. Design Language 13
2. Application Shell 13
3. Key Screens 13
4. Core Interactions 14
5. Responsive Behaviour 15

04 Deployment & Operations 16
1. Self-Hosting with Docker Compose 16
2. Environment Variables 17
3. Reverse Proxy / TLS 18
4. Local Development 18
5. Backups & Disaster Recovery 19
6. Monitoring & Health 19

05 Delivery Roadmap 20
Phase 1 — MVP (Weeks 1–6) 20
Phase 2 — Team Collaboration (Weeks 7–12) 20
Phase 3 — Scale & Ecosystem (Quarter 2+) 20
Phase 4 — AI & Automation (Future) 21
Assumptions 21
Open Questions for the Client 21

00

ProjectBoard — Product Overview

> **ProjectBoard** is a placeholder product name used throughout this document set. It is intended to be replaced with the client's chosen brand before delivery.

1. What ProjectBoard Is

ProjectBoard is a modern, web-based **Kanban-style project management system** — a self-hostable alternative to commercial tools such as Trello. It gives teams a visual way to organise work into boards, lists, and cards, and to collaborate in real time around shared tasks.

2. Business Value

- **Own your data** — fully self-hosted; no third-party SaaS dependency, no per-seat SaaS pricing.
- **Fast to deploy** — ships as Docker images with a one-command compose setup, including database provisioning and automatic migrations.
- **Familiar paradigm** — the Kanban board model requires no training for users coming from Trello, Jira, or sticky notes.
- **White-label ready** — branding, colours, and naming are fully configurable for the client.
- **Extensible** — public REST API with API keys, webhooks, and an AI-assistant integration surface (MCP-compatible) for future automation.

3. Who It Serves

AUDIENCE	BENEFIT
Small–mid product/engineering teams	Lightweight sprint and task tracking
Agencies & consultancies	Client-facing project boards with public share links
IT departments (internal tools)	Self-hosted, domain-restricted login (OIDC/social SSO)
Operations / marketing teams	Editorial calendars, campaign tracking via templates

4. High-Level Scope

In scope (v1):

- Workspace & member management with role-based permissions
- Boards with configurable visibility (private / workspace / public read-only)
- Lists and cards with drag-and-drop reordering, due dates, labels, members, checklists
- Comments and per-card activity history

- Reusable board templates
- Import of existing Trello boards
- Email/password authentication plus optional Google / GitHub / Discord / generic OIDC login
- Email notifications via any SMTP server (optional)
- File attachments on cards via S3-compatible object storage (optional)
- Public REST API (`/api/v1`) with per-user API keys
- Webhooks for workspace events
- Docker Compose self-hosting with automated database migrations

Deferred (future phases):

- Third-party integrations marketplace (GitHub etc. as native card integrations)
- Subscription/billing module (Stripe) if offered as a managed service
- AI assistant control via an MCP-compatible server (foundation already present)

5. Document Set

DOCUMENT	CONTENTS
01 — Requirements	Functional and non-functional requirements
02 — Architecture	System design, tech stack, data model
03 — UI / UX	Screens, layout, interaction design
04 — Deployment	Docker hosting, environment variables, operations
05 — Roadmap	Phased delivery plan and open questions

01

ProjectBoard — Requirements

This document defines the functional and non-functional requirements for ProjectBoard. Requirement IDs (FR-x, NFR-x) are stable identifiers for use in proposals, contracts, and acceptance testing.

1. Functional Requirements

1.1 Boards, Lists & Cards

ID	REQUIREMENT
FR-1.1	Users can create, rename, archive, and delete boards.
FR-1.2	Each board contains ordered lists (columns); lists can be created, renamed, reordered, and deleted.
FR-1.3	Each list contains ordered cards; cards can be created, edited, reordered within a list, and moved between lists via drag-and-drop.
FR-1.4	Cards support: title, rich-text description, due date, cover/styling, member assignment, and labels.
FR-1.5	Cards carry a per-board sequential card number for easy reference (e.g. in commits or chat).
FR-1.6	Cards support checklists with individually toggleable items.
FR-1.7	Cards support file attachments when object storage is configured.
FR-1.8	Users can duplicate a card, and watch/subscribe to card changes.
FR-1.9	Archived boards and cards are soft-deleted (recoverable).
FR-1.10	Boards can be starred as favourites by each user.

1.2 Visibility & Sharing

ID	REQUIREMENT
FR-2.1	Board visibility levels: **Private** (invited members only), **Workspace** (all workspace members), **Public** (anyone with the link, read-only).
FR-2.2	Public boards render on a clean shareable URL that requires no login and hides vendor branding (white-label).
FR-2.3	Board access is enforced server-side on every API call, not only in the UI.

1.3 Workspaces, Members & Roles

ID	REQUIREMENT
FR-3.1	Users belong to one or more workspaces; each workspace has a unique human-readable slug.
FR-3.2	Workspace owners/admins can invite members by email or via invite links.

FR-3.3	Role-based access control: at minimum Owner, Admin, Member, and Viewer roles, with configurable per-role permissions (custom roles supported).
FR-3.4	Membership can be paused (retains history, removes access).
FR-3.5	Invited members can be assigned to boards and to individual cards.

1.4 Collaboration

ID	REQUIREMENT
FR-4.1	Users can comment on cards; comments support editing and deletion by their author (and workspace admins).
FR-4.2	Every change to a card (title, description, moves, label/member changes, checklist progress, attachments, comments) is recorded in an immutable **activity log** visible on the card.
FR-4.3	Users receive in-app notifications for relevant events (mentions, assignments, comments on watched cards).
FR-4.4	Email notifications are sent when SMTP is configured; all email features can be disabled globally.
FR-4.5	Users can submit feedback from inside the app.

1.5 Organisation & Discovery

ID	REQUIREMENT
FR-5.1	Colour-coded labels, scoped per board; cards can carry multiple labels.
FR-5.2	Board filtering: by label, member, due date, and free-text search.
FR-5.3	A global command palette (keyboard-driven quick actions/navigation).
FR-5.4	Board templates: any board can be saved as a template and instantiated later.

1.6 Import / Export

ID	REQUIREMENT
FR-6.1	One-click import of Trello boards (lists, cards, labels, members, comments, attachments) via Trello API credentials configured by the administrator.
FR-6.2	Import jobs run in the background with a visible status per import.
FR-6.3	Public REST API (`/api/v1` , OpenAPI-documented, Bearer API keys) for programmatic access to workspaces, boards, lists, cards, comments, checklists, labels, and members.

1.7 Automation Surface

ID	REQUIREMENT
FR-7.1	Workspace-scoped webhooks: outbound HTTP callbacks for board/card events, with delivery logging and retry.

FR-7.2	(Phase 2) MCP-compatible server exposing the API to AI assistants via natural language, using the same API keys.

1.8 Authentication

ID	REQUIREMENT
FR-8.1	Email + password registration and login; password reset via email.
FR-8.2	Optional social login: Google, GitHub, Discord.
FR-8.3	Optional enterprise login via any OIDC-compliant identity provider, with an allow-list of permitted email domains.
FR-8.4	Session management with secure, httpOnly cookies; session records are server-side revocable.
FR-8.5	Sign-up can be disabled globally (invite-only deployments).
FR-8.6	Per-user API keys for programmatic access, created and revoked from user settings.

1.9 Administration

ID	REQUIREMENT
FR-9.1	Per-user settings: profile, avatar (stored in object storage when configured), password, API keys, notification preferences.
FR-9.2	Workspace settings: name, slug, members, roles & permissions, invite links, webhooks, integrations, billing (if enabled).
FR-9.3	Instance-level admin endpoints protected by a server-only admin API key (statistics, health).
FR-9.4	Rate limiting on sensitive endpoints (email login, sign-up), backed by Redis when configured, in-memory otherwise.

2. Non-Functional Requirements

ID	REQUIREMENT
NFR-1	**Self-hostable**: the entire product runs from Docker images on a single host; no external SaaS dependency. Optional externals (SMTP, S3, Redis, OAuth providers) are strictly optional.
NFR-2	**Reliable upgrades**: database migrations run automatically and idempotently before application start (run-once migration container).
NFR-3	**Security**: parameterized queries throughout (ORM), server-side authorisation on every endpoint, CSRF protection, secure cookie configuration, TLS assumed at the reverse proxy, rate limiting on auth endpoints, upload size/type restrictions (avatars limited to 2 MB).
NFR-4	**Performance**: board view loads in under 2 s on broadband for boards with 1,000+ cards; drag-and-drop interactions remain at 60 fps (client-side optimistic updates).
NFR-5	**Scalability**: stateless web tier (horizontal scaling ready); state held in PostgreSQL and optional Redis.

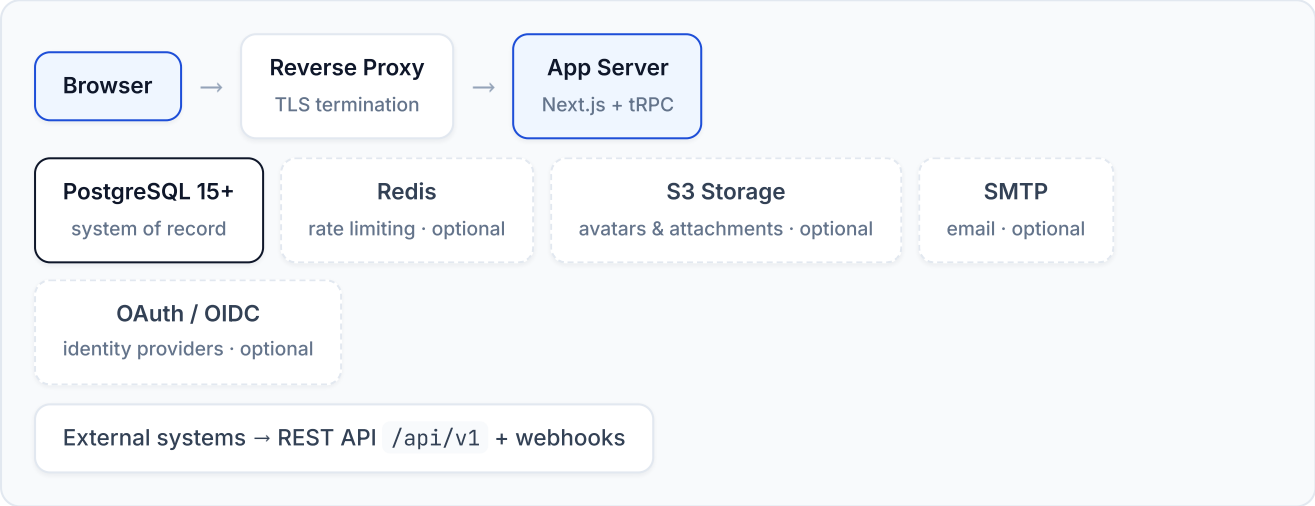
NFR-6	**Availability**: graceful degradation — email/S3/Redis failures disable only the dependent features, never the core boards experience.
NFR-7	**Localisation-ready**: UI strings externalised (English shipped; additional languages can be added without code changes).
NFR-8	**Observability**: structured logs with configurable verbosity; health-check endpoint for container orchestration.
NFR-9	**Backup-friendly**: all persistent state in one PostgreSQL database plus an S3 bucket — standard tooling applies.
NFR-10	**Browser support**: current versions of Chrome, Edge, Firefox, Safari; responsive down to tablet widths.

02

ProjectBoard — System Architecture

1. High-Level Design

ProjectBoard is a **full-stack web application** built on a single deployable unit: a Next.js application server backed by PostgreSQL, with optional external services (SMTP, S3-compatible storage, Redis) that are strictly optional.



Deployment topology (Docker Compose): a run-once `migrate` container applies database migrations, then the `web` container starts, backed by a `postgres` service with a health check. All services sit on a private bridge network; only the web port is published.

2. Technology Stack

LAYER	CHOICE	RATIONALE
Frontend & server	**Next.js** (pages router), React, TypeScript	One framework for UI and API; server-side rendering for fast first paint and SEO-friendly public boards.
API layer	**tRPC** (end-to-end typesafe RPC) plus an auto-generated **OpenAPI REST layer** (`/api/v1`) for external consumers	Type safety across client/server; public API for integrations without tRPC coupling.
Styling	**Tailwind CSS** + shadcn-style component primitives	Consistent, minimal design system; fast iteration.
Animation / DnD	framer-motion, **@dnd-kit**	Accessible, performant drag-and-drop with keyboard support.
Rich text	Lightweight rich-text editor component for card descriptions and comments	Formatted task descriptions without heavyweight CMS dependencies.

Database / ORM	**PostgreSQL** + **Drizzle ORM** with timestamped SQL migrations (Drizzle Kit)	Type-safe schema-as-code; migrations are plain SQL reviewed in version control.
Auth	**Better Auth** (email/password, social, OIDC, API keys, sessions)	Single integrated session/auth layer with server-side session records.
Email	**React Email** templates over any SMTP server	Provider-independent; works with internal relays or services like Resend.
i18n	Externalised message catalogues (e.g. Lingui)	English shipped; more languages added without code changes.
Monorepo tooling	pnpm workspaces + Turborepo; shared ESLint/Prettier/TS configs across `apps/*` and `packages/*`	Clean separation of web app, API package, DB package, auth, email.
Testing	Vitest (API integration tests), Playwright (end-to-end)	

Suggested monorepo layout

```
repo/
├── apps/
│   └── web/            # Next.js application (pages, components, Dockerfile)
├── packages/
│   ├── api/            # tRPC routers (board, card, list, label, member,
│   │                   #   workspace, permission, attachment, checklist,
│   │                   #   webhook, import, integration, feedback, health)
│   ├── db/             # Drizzle schema, migrations/, repository layer, Redis client
│   ├── auth/           # Better Auth configuration
│   ├── email/          # React Email templates
│   └── shared/         # shared types & utilities
└── docker-compose.yml
```

3. Data Model (core entities)

Relational schema (~30 tables). Key entities and relationships:

```
user ──< session, account, verification, apikey (auth tables)
user >── workspace_members ──< workspace ──< workspace_invite_links
workspace ──< boards ──< list ──< cards ──< card_comments
                                        ├──< card_activity      (typed, immutable)
                                        ├──< card_attachment
                                        ├──< cards_to_labels >── label
                                        ├──< cards_to_workspace_members
                                        ├──< card_checklist ──< card_checklist_item
                                        └── card numbers (per-board sequence)
workspace ──< workspace_roles ──< workspace_role_permissions
workspace ──< workspace_member_permissions
workspace ──< workspace_webhooks ──< webhook_events
workspace ──< imports (e.g. Trello import jobs, with status)
user >──────< notifications
user_board_favorites (user × board)
```

Design notes:

- **Activity is append-only** (`card_activity` with a typed event enum) — powers both the card history UI and audit requirements.
- **Soft deletion** on boards/cards via `isArchived` flags.
- **Custom roles** are a role → permission mapping, overridable per member.
- **Slugs** give every workspace/board a human-readable URL (`/<workspaceSlug>/<boardSlug>`), checked for uniqueness at creation.

4. API Surface

SURFACE	LOCATION	PURPOSE
Internal RPC	`/api/trpc/*` (tRPC)	All UI operations — type-safe, session-authenticated
Public REST	`/api/v1` (OpenAPI spec at `/api/v1/openapi.json`)	External integrations; Bearer API keys (`pbk_…` style)
Auth	`/api/auth/*`	Better Auth handlers (login, OAuth/OIDC callbacks, password reset)
Uploads	`/api/upload/{avatar,attachment}` , `/api/download/attachment`	Direct-to-server then S3 (or local fallback)
Webhooks	outbound, workspace-scoped	Event delivery with per-endpoint event log & retry
Admin	server-only admin API key	Instance statistics and maintenance endpoints
AI assistant (Phase 2)	MCP-compatible server (npx-distributable or in-app endpoint)	Natural-language control over the same API surface

5. Security Architecture

- **Authorisation is enforced in the API layer** — every tRPC procedure and REST route re-checks board/workspace membership and visibility; the UI never trusts client state.
- Parameterised queries exclusively via the ORM; no raw string interpolation.
- Sessions: httpOnly, SameSite cookies; server-side session table allows revocation.
- Rate limiting on login/sign-up (Redis when available, in-memory fallback).
- Upload restrictions: file-type allow-list, avatar size cap (2 MB), configurable request body limit.
- Secrets via environment variables only; no secrets in the client bundle (`NEXT_PUBLIC_*` naming convention strictly reviewed).
- Deployment assumes TLS termination at a reverse proxy (nginx/Traefik/cloud LB).

6. Build & Release

- **Web image**: multi-stage build — dependency pruning (`turbo prune`), `pnpm install` , Next.js *standalone* output, final stage on a **distroless Node base** (minimal attack surface), port 3000. A small bootstrap script injects runtime environment variables so one image serves any environment.
- **Migrate image**: separate lightweight run-once image executing `drizzle-kit migrate` against the configured `POSTGRES_URL` .
- CI runs lint, type-check, unit/integration tests (Vitest) and end-to-end tests (Playwright) before image publication.

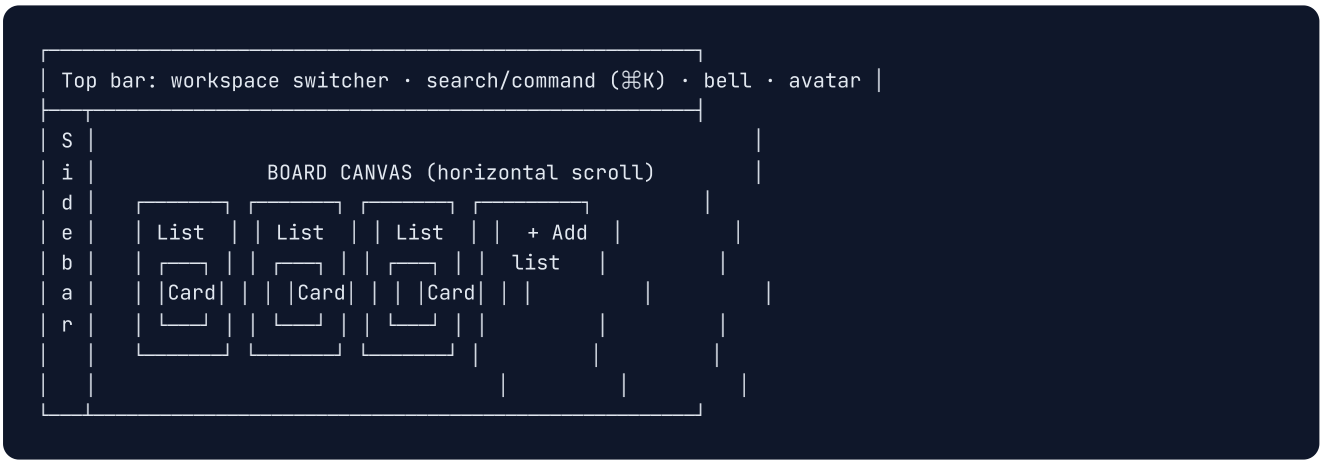
03

ProjectBoard — UI / UX Specification

1. Design Language

- **Aesthetic**: clean, minimal, content-first. Boards sit on subtle background textures/gradients; cards are white with soft shadows and rounded corners.
- **Component system**: Tailwind CSS with a shadcn-style primitive set — consistent spacing, typography, and focus states across every screen.
- **Motion**: purposeful micro-animations (card hover lift, smooth list reordering, modal transitions) via framer-motion. Animations respect reduced-motion settings.
- **Accessibility**: drag-and-drop has keyboard-operable alternatives; ARIA roles on dialogs, menus, and the command palette; visible focus rings throughout.
- **White-label**: product name, logo, and landing copy are configuration-driven; public shared boards can hide all vendor attribution.

2. Application Shell



Left sidebar: workspaces → boards (with per-user favourites at top), link to templates, workspace member list. Collapsible on smaller screens.

3. Key Screens

SCREEN	DESCRIPTION
Landing	Product marketing page with feature highlights and a call to action.
Login / Sign-up	Email+password form plus optional social/OIDC buttons.
Onboarding	Two short steps: choose a plan (if billing enabled) → create first workspace.
Boards home (`/boards`)	All boards across the user's workspaces; favourites section.

Board view	The core screen (see §2). List lanes scroll horizontally; each lane has a header (title, card count) and a composer at the bottom ("Add a card…"). Filters bar: label chips, member avatars, due-date presets, search box.
Card detail	Opens as a focused page/route (deep-linkable, e.g. `/cards/<id>`): title (inline edit), list/position indicator, members & labels in the header row, rich-text description, checklist(s) with progress bars, attachments grid, comment thread (newest last), and a right-hand rail with **Activity** (immutable, timestamped) and card metadata (due date, watchers, card number).
Public shared board (`/<workspaceSlug>/<boardSlug>`)	Read-only rendering of a public board on a clean, brand-free URL — no chrome, no login prompt.
Workspace dashboard (`/<workspaceSlug>`)	Workspace overview: boards, members, recent activity.
Members	Invite by email, manage invite links, assign roles, pause members.
Templates	Template gallery; "use template" creates a new board pre-populated with its lists/cards/labels.
Settings (account & workspace)	Profile & avatar, password, API keys, notification preferences · workspace name/slug, roles & permissions editor, webhooks (with delivery log), integrations, billing.
Invite accept (`/invite/<code>`)	Signed-in flow to join a workspace via link.
Upgrade / pricing	Shown only when the billing module is enabled.
Feedback modal	Reachable from the user menu; submits to a feedback store for the operators.

4. Core Interactions

- **Drag & drop** (@dnd-kit): cards reorder within/across lists; whole lists reorder on the board. Optimistic UI — the board updates instantly and syncs in the background; failures roll back with a toast.
- **Card composer**: single-click "Add a card" keeps focus for rapid entry (type → Enter → next card); Esc dismisses.
- **Inline editing**: click card title, list title, or board title to edit in place.
- **Filters**: combine label + member + due-date + text; filtered-out cards dim or hide (toggle).
- **Command palette** (`⌘K` / `Ctrl+K`): fuzzy jump to boards, quick actions (new board, new card, go to settings).
- **Due dates**: colour-coded badges (soon = amber, overdue = red); optional date-picker with time.
- **Checklists**: per-card, add items inline, progress ring/bar on the card face.
- **Comments**: markdown-flavoured rich text, avatar + timestamp header, edit/delete on own comments.
- **Notifications**: bell dropdown with unread counts; deep-links to the relevant card.

5. Responsive Behaviour

- Desktop-first board canvas; on tablets, lists keep full width and the sidebar collapses to an icon rail.
- Below tablet widths the app shows a friendly "desktop recommended" notice (Kanban boards are inherently wide); public shared boards remain fully responsive since they are read-only.

04

ProjectBoard — Deployment & Operations

1. Self-Hosting with Docker Compose

The standard deployment is a three-service Compose stack. A **run-once migrate container** applies database migrations before the application starts, so upgrades are a single `docker compose up -d`.

```
services:
  migrate:
    image: <registry>/projectboard-migrate:latest
    container_name: pb-migrate
    networks: [pb-network]
    environment:
      - POSTGRES_URL=${POSTGRES_URL}
    depends_on:
      postgres:
        condition: service_healthy
    restart: "no"

  web:
    image: <registry>/projectboard:latest
    container_name: pb-web
    ports:
      - "${WEB_PORT:-3000}:3000"
    networks: [pb-network]
    env_file: [.env]
    depends_on:
      migrate:
        condition: service_completed_successfully
    restart: unless-stopped

  postgres:
    image: postgres:15
    container_name: pb-db
    environment:
      - POSTGRES_DB=projectboard
      - POSTGRES_USER=projectboard
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    volumes:
      - pb_postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U projectboard -d projectboard"]
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped
    networks: [pb-network]

networks:
  pb-network:

volumes:
  pb_postgres_data:
```

Bring-up: `docker compose up -d` → the app is available at `http://localhost:3000` (or `WEB_PORT`).

Day-2 commands:

TASK	COMMAND
Stop	`docker compose down`
Logs	`docker compose logs -f [web|migrate|postgres]`
Restart	`docker compose restart`
Upgrade (after pulling new images)	`docker compose up -d` (migrations re-run automatically)

The `web` image is built multi-stage (dependency prune → standalone Next.js build → distroless Node runtime) and injects runtime environment variables at container start, so **one image serves dev, staging, and production** with only `.env` differing.

2. Environment Variables

VARIABLE	PURPOSE	REQUIRED
`POSTGRES_URL`	PostgreSQL connection URL	Yes (or external DB)
`NEXT_PUBLIC_BASE_URL`	Public URL of the installation (links, auth callbacks)	Yes
`BETTER_AUTH_SECRET`	Auth encryption secret — random 32+ chars	Yes
`BETTER_AUTH_TRUSTED_ORIGINS`	Allowed callback origins	No
`NEXT_PUBLIC_ALLOW_CREDENTIALS`	Enable email/password login	Yes
`NEXT_PUBLIC_DISABLE_SIGN_UP`	Close public registration (invite-only)	No
`REDIS_URL`	Rate limiting backend	No (in-memory fallback)
`EMAIL_FROM` / `SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASSWORD` / `SMTP_SECURE`	Outbound email	For email features
`NEXT_PUBLIC_DISABLE_EMAIL`	Globally disable all email features	No
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`	Google login	No
`GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET`	GitHub login	No
`DISCORD_CLIENT_ID` / `DISCORD_CLIENT_SECRET`	Discord login	No
`OIDC_CLIENT_ID` / `OIDC_CLIENT_SECRET` / `OIDC_DISCOVERY_URL`	Generic enterprise SSO	No

`BETTER_AUTH_ALLOWED_DOMAINS`	Domain allow-list for SSO logins	No
`S3_REGION` / `S3_ENDPOINT` / `S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY` / `S3_FORCE_PATH_STYLE`	S3-compatible storage for avatars/attachments	For uploads
`NEXT_PUBLIC_AVATAR_BUCKET_NAME` / `NEXT_PUBLIC_ATTACHMENTS_BUCKET_NAME`	Bucket names	For uploads
`S3_AVATAR_UPLOAD_LIMIT`	Avatar size cap (default 2 MB)	No
`NEXT_PUBLIC_STORAGE_URL`	Storage CDN/service URL	For uploads
`TRELLO_APP_API_KEY` / `TRELLO_APP_API_SECRET`	Trello board import	For import
`NEXT_PUBLIC_WHITE_LABEL_HIDE_POWERED_BY`	Hide vendor attribution on public boards	No
`PB_ADMIN_API_KEY`	Instance admin/stats endpoints	Recommended
`NEXT_API_BODY_SIZE_LIMIT`	Max request body (default 1 MB)	No
`LOG_LEVEL`	`debug` / `info` / `warn` / `error`	No

3. Reverse Proxy / TLS

Terminate TLS in front of the container (nginx, Traefik, Caddy, or a cloud load balancer). Example nginx server block:

```
server {
    listen 443 ssl;
    server_name boards.example.com;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

4. Local Development

```
pnpm install                # monorepo deps
cp .env.example .env        # configure variables
pnpm db:migrate             # apply migrations (Drizzle Kit)
pnpm dev                    # start dev server
```

Tests: `pnpm test` (Vitest API integration) · `pnpm e2e` (Playwright).

5. Backups & Disaster Recovery

- **Database** (mandatory): nightly `pg_dump` of the `projectboard` database; point-in-time recovery if WAL archiving is enabled. A restore drill should be scheduled quarterly.
- **Object storage** (if enabled): bucket versioning/replication for the avatars and attachments buckets.
- **Secrets**: keep `.env` under version-controlled encryption (e.g. sops/age); rotate `BETTER_AUTH_SECRET` and `PB_ADMIN_API_KEY` on staff changes.
- **Recovery target**: RPO ≤ 24 h (nightly dump), RTO ≤ 2 h (restore dump + `docker compose up -d`).

6. Monitoring & Health

- Container health check: `pg_isready` on the database; app exposes a health endpoint for the web tier.
- Structured JSON logs with `LOG_LEVEL`; ship via the container runtime's log driver.
- Optional: admin API key unlocks instance statistics endpoints for uptime monitoring.

05

ProjectBoard — Delivery Roadmap

Phase 1 — MVP (Weeks 1–6)

Goal: a working, self-hosted boards product that a pilot team can use daily.

- Authentication: email/password, password reset, sessions
- Workspaces, member invitations, base roles (Owner / Admin / Member / Viewer)
- Boards / lists / cards with drag-and-drop (FR-1.1 – FR-1.4)
- Board visibility: private / workspace / public share links
- Labels, due dates, members on cards
- Comments + full card activity log
- Board and card archiving, favourites
- Docker Compose deployment with auto-migrations
- Landing, login/sign-up, onboarding, board view, card detail

Exit criteria: a pilot workspace of 10 users runs real work on it for two weeks with zero data-loss incidents; `docker compose up -d` from a clean machine reaches a working app in under 15 minutes.

Phase 2 — Team Collaboration (Weeks 7–12)

- Checklists on cards
- Board filters (label / member / due date / text)
- Command palette
- Board templates (save & instantiate)
- In-app notifications + email notifications (SMTP)
- Avatars & attachments via S3-compatible storage
- Public REST API (`/api/v1`) with user API keys + OpenAPI docs
- Workspace webhooks with delivery log
- Custom roles & per-member permission overrides
- Optional social/OIDC login (Google, GitHub, Discord, enterprise OIDC)
- Rate limiting (Redis) and instance admin endpoints

Exit criteria: FR coverage for sections 1.1–1.6, 1.8, 1.9 of the requirements doc verified by automated test suites (unit + integration + e2e).

Phase 3 — Scale & Ecosystem (Quarter 2+)

- Trello import (background jobs with status tracking)
- i18n: ship ≥ 2 additional languages
- Billing module (Stripe) if offered as a managed/SaaS variant

- Horizontal scaling runbook (stateless web tier behind a load balancer, managed Postgres)
- Integrations marketplace groundwork (GitHub-style native card integrations)

Phase 4 — AI & Automation (Future)

- MCP-compatible server exposing the full API to AI assistants (Claude Desktop, Cursor, Copilot, …), distributed via `npx`, authenticated with existing user API keys
- Suggested automation rules (e.g. "when card moves to Done → notify watchers")

Assumptions

1. Client provides (or approves) hosting: a single VM with Docker, or an internal Kubernetes cluster; TLS certificate.
2. Client decides at kickoff: open sign-up vs invite-only; which SSO providers; whether email notifications are needed at launch.
3. Placeholder branding ("ProjectBoard") is replaced with the client's brand before Phase 1 exit; white-label settings hide vendor attribution on public boards.
4. Data residency stays fully on client infrastructure unless a managed variant is contracted.

Open Questions for the Client

#	QUESTION	AFFECTS
1	Do users need to log in with an existing corporate identity provider (OIDC/ADFS)?	Phase 2 auth scope
2	Should public share links exist at all, or are boards internal-only?	Security review, FR-2
3	Any compliance regime (GDPR, HIPAA, ISO 27001) to design documentation around?	Hosting & audit features
4	Expected concurrent users / board sizes?	Sizing, NFR-4/5 targets
5	Is a SaaS/billing variant wanted later, or strictly self-hosted?	Phase 3 scope
6	Preferred language(s) at launch?	i18n schedule